



PRÉ-RENTREE 2026-2027

SÉANCE 2 - BOUCLES, COMPTEURS ET ACCUMULATEURS

Dossier personnalisé - Ahmed BENHADJ SALEM - durée : 2 heures

Parcours : Fiabilisation et autonomie

Objectif personnel de la séance 2 : distinguer précisément **for** et **while**, sécuriser les accumulateurs, traiter les cas limites et justifier un **invariant** ainsi qu'un **variant** simple.

Production à conserver : le fichier `S2_boucles_Ahmed.py`, réalisant en un seul parcours l'analyse fiable d'une série de mesures, avec tests prévus avant le code et comportement explicite sur liste vide.

Contrat de fiabilisation

1. Je transforme la spécification en jeux de tests **avant** de coder.
2. Je distingue à chaque tour : nombre d'itérations, somme produite et état courant.
3. Je justifie l'initialisation d'un accumulateur ou d'un extremum.
4. Je traite explicitement le cas vide avant d'utiliser le premier élément.
5. Dans une boucle **while**, je nomme ce qui évolue et pourquoi l'arrêt est garanti.

1. Feuille de route des 120 minutes

Temps	Travail	Preuve attendue
0–10 min	Rituel chronométré : range , compteur et somme.	Prévision exacte et distinction des états.
10–30 min	Bornes, indices et audit de boucles.	Trois bugs diagnostiqués.
30–50 min	Accumulateur, extremum et invariant.	Initialisations justifiées.
50–65 min	Choix for/while , terminaison et variant.	Preuve d'arrêt simple.
65–70 min	Pause.	–
70–100 min	Atelier principal : analyse fiable en un seul parcours.	Code exécutable et cas vide traité.
100–112 min	Parcours Ahmed : audit, tests de seuil et boucle bornée par essais.	Correction minimale documentée.
112–120 min	Invariant, revue et exit ticket.	Explication concise et contrôlée.

Critères de réussite : somme correcte et justifiée ; tests normaux et limites exécutés ; aucune confusion entre effectif et somme ; comportement défini pour la liste vide ; variant identifiable dans la boucle **while**.

2. Rituel de fiabilité - 10 min

Fichier à créer : S2_boucles_Ahmed.py. Objectif : prévision précise, pas de réponse intuitive non contrôlée.

Rituel 1 - range exact

Sans exécuter, écrire les valeurs produites par :

```
range(2, 9, 3)
```

Valeurs : _____

Nombre de tours : _____

Certitude : ☐ 1 ☐ 2 ☐ 3 ☐ 4

Contrôle : _____

Rituel 2 - somme et effectif ne sont pas la même chose

```
total = 0
effectif = 0

for i in range(1, 4):
    total += i
    effectif += 1
```

Compléter sans exécuter :

Tour	i	total avant	total après	effectif après
1				
2				
3				

Valeur finale de **total** : _____

Valeur finale de **effectif** : _____

Rituel 3 - contrôle rapide

Donner deux méthodes différentes pour contrôler la somme obtenue, sans relire le code ligne par ligne.

Point de vigilance. Une boucle peut effectuer trois tours sans que la somme accumulée vaille 3. Le nombre de tours décrit l'effectif ; l'accumulateur dépend des valeurs ajoutées.

3. Bornes, indices et audit de boucles - 20 min

A. Produire exactement les valeurs demandées

Valeurs attendues	range proposé	Justification de la borne exclue
2, 5, 8		
0, 1, ..., n-1		
10, 8, 6, 4		
5, 4, 3, 2, 1		

B. Audit 1 - accès hors liste

```
L = [10, 20, 30]
for i in range(len(L) + 1):
    print(L[i])
```

1. Valeurs successives de i : _____
2. Premier indice qui provoque une erreur : _____
3. Correction minimale : _____
4. Jeu de test le plus court qui révèle le bug : _____

C. Audit 2 - élément oublié

```
for i in range(1, len(L)):
    print(L[i])
```

Quel élément n'est jamais traité? _____

Pourquoi?

D. Audit 3 - mauvais pas

Le code veut parcourir 10, 8, 6, 4.

```
for i in range(10, 3, 2):
    print(i)
```

Prévoir ce qui se passe, puis proposer une correction minimale.

Aides graduées - une seule à la fois : A - écrire les valeurs de `range` ; B - nommer le schéma (compter, additionner, chercher, répéter) ; C - initialiser avant la boucle ; D - vérifier ce qui évolue ; E - compter les tours et comparer avec un calcul manuel.

4. Accumulateurs et extrema fiables - 20 min

A. Trois bugs à diagnostiquer

Bug 1 - le code compte au lieu d'additionner

```
total = 0
for valeur in mesures:
    total += 1
```

Symptôme : _____

Correction : _____

Bug 2 - l'accumulateur est réinitialisé

```
for valeur in mesures:
    total = 0
    total += valeur
```

Pourquoi le résultat final ne représente-t-il pas la somme ?

Bug 3 - maximum faux sur une liste négative

```
maximum = 0
for valeur in [-5, -2, -9]:
    if valeur > maximum:
        maximum = valeur
```

Prévoir le résultat actuel, donner le résultat attendu et justifier une initialisation robuste.

B. État conservé après chaque tour

Pour une liste non vide, compléter la signification des variables après k éléments traités.

Variable	Propriété qui doit rester vraie
total	
effectif	
minimum	
maximum	
nb_seuil	

Invariant à viser. Après chaque tour, les variables résument exactement le préfixe de la liste déjà parcouru, et aucun élément futur n'a encore été utilisé.

5. for, while, terminaison et variant - 15 min

A. Choix de boucle argumenté

Situation	Boucle	Justification technique
Parcourir un fichier déjà chargé en mémoire		
Répéter exactement 20 simulations		
Demander une saisie jusqu'à ce qu'elle soit valide		
Chercher dans une liste en s'arrêtant dès la cible trouvée		
Effectuer au plus trois tentatives de mot de passe		

B. Boucle avec variant explicite

Le programme doit autoriser au plus trois essais.

```
essais_restants = 3
mot_de_passe_correct = False

while _____:
    # lire et verifier une proposition

    if _____:
        mot_de_passe_correct = True
    else:
        _____
```

1. Écrire une condition qui garantit au plus trois essais.
2. Nommer le variant : _____
3. Donner ses valeurs possibles : _____
4. Expliquer pourquoi il ne peut pas décroître indéfiniment.

5. Prévoir les tests : réussite au premier essai, réussite au troisième, trois échecs.

Test	Scénario	Sortie ou état attendu	OK
1			☐
2			☐
3			☐

6. Atelier principal - spécifier et tester - 30 min

Spécification. Écrire un programme qui analyse une liste de mesures en **un seul parcours** et calcule : somme, effectif, moyenne, minimum, maximum et nombre de valeurs supérieures ou égales à un seuil. Le comportement sur liste vide doit être explicite.

Données de référence :

```
mesures = [18, 21, -2, 27, 31, 15]
seuil = 25
```

Étape 1 - tests AVANT code

Compléter tous les résultats attendus avant l'implémentation.

Test	Liste	somme	effectif	moyenne	min / max	nb $\geq$ seuil
1	liste de référence					
2	[5]					
3	[-4, -9, -2]					
4	[]					

Étape 2 - décision sur le cas vide

Choisir et justifier un contrat parmi les deux possibilités suivantes :

- afficher un message et ne pas calculer de moyenne ni d'extrema ;
- attribuer une valeur sentinelle explicite aux résultats non définis.

Décision retenue : _____

Étape 3 - plan d'état

Variable	Initialisation	Justification
total		
effectif		
minimum		
maximum		
nb_seuil		

7. Atelier principal - implémentation autonome

Contraintes : un seul parcours de la liste ; pas de `sum`, `min` ni `max` ; cas vide traité avant `mesures[0]` ; noms explicites ; résultats comparés aux tests prévus.

```
mesures = [18, 21, -2, 27, 31, 15]
seuil = 25

if len(mesures) == 0:
    # comportement choisi pour le cas vide
    -----
else:
    total = -----
    effectif = -----
    minimum = -----
    maximum = -----
    nb_seuil = -----

    for valeur in mesures:
        total = -----
        effectif = -----

        if valeur < minimum:
            -----

        if -----:
            maximum = valeur

        if -----:
            nb_seuil = -----

    moyenne = -----

    print("somme :", total)
    print("effectif :", effectif)
    print("moyenne :", moyenne)
    print("minimum :", minimum)
    print("maximum :", maximum)
    print("nombre au-dessus du seuil :", nb_seuil)
```

Revue de code

1. La liste est-elle parcourue exactement une fois ? ☐oui ☐non
2. Le premier élément est-il utilisé uniquement après le test du cas vide ? ☐oui ☐non
3. L'effectif est-il bien un compteur et la somme un accumulateur ? ☐oui ☐non
4. Chaque condition est-elle couverte par un test ? ☐oui ☐non
5. Expliquer en trois phrases l'état du programme après le troisième tour.

8. Parcours personnalisé Ahmed - 12 min

A. Cas limite : seuil atteint exactement

Ajouter un test contenant une valeur exactement égale au seuil. Écrire le résultat attendu et expliquer le choix de `>=` ou `>`.

B. Correction minimale - une seule modification à la fois

Le programme suivant doit compter les valeurs positives et calculer leur somme.

```
valeurs = [-2, 4, 7, -1, 3]
nb_positives = 0
somme_positives = 0

for valeur in valeurs:
    if valeur >= 0:
        nb_positives += 1
        somme_positives += valeur

print(nb_positives, somme_positives)
```

1. Écrire la spécification exacte en français : _____
2. Prévoir la sortie actuelle : _____
3. Identifier les deux écarts entre spécification et code.

4. Corriger sans réécrire tout le programme.

C. Défi en un parcours

Ajouter au programme principal le calcul de l'amplitude maximum – minimum sans parcourir une seconde fois la liste. Justifier l'endroit où le calcul doit être placé.

9. Invariant, tests et journal de débogage - 10 min

A. Invariant de l'analyse

Compléter précisément : après k tours, pour les k premières valeurs déjà parcourues,

- `effectif` = _____ ;
- `total` = _____ ;
- `minimum` et `maximum` sont _____ ;
- `nb_seuil` est _____.

Pourquoi cet invariant est-il vrai avant le premier tour ?

Pourquoi reste-t-il vrai après le traitement d'une nouvelle valeur ?

B. Tableau d'exécution des tests

Test	Entrée	Attendu	Obtenu	OK
1	liste de référence			☐
2	liste à un élément			☐
3	liste entièrement négative			☐
4	liste vide			☐
5	valeur égale au seuil			☐

C. Journal de débogage

Symptôme	Hypothèse	Modification unique	Résultat
----------	-----------	---------------------	----------

Aides graduées - une seule à la fois : A - écrire les valeurs de `range` ; B - nommer le schéma (compter, additionner, chercher, répéter) ; C - initialiser avant la boucle ; D - vérifier ce qui évolue ; E - compter les tours et comparer avec un calcul manuel.

10. Trace écrite, exit ticket et auto-évaluation - 8 min

Trace écrite à conserver

- `range` exclut la borne de fin ; les indices valides d'une liste de longueur `n` vont de 0 à `n-1`.
- Un compteur mesure un effectif ; un accumulateur construit une valeur à partir des données rencontrées.
- Un extremum sur liste non vide s'initialise avec le premier élément, pas avec une constante arbitraire.
- Un invariant décrit ce qui reste vrai avant et après chaque tour ; un variant évolue vers l'arrêt d'une boucle `while`.
- Les cas limites sont écrits avant le code et conservés comme preuve de fiabilité.

Exit ticket - sans aide

1. Écrire les valeurs produites par `range(10, 3, -2)`.

Certitude : ☐1 ☐2 ☐3 ☐4

Contrôle : _____

2. Expliquer pourquoi trois tours ne signifient pas que la somme vaut 3.

3. Donner l'invariant d'un accumulateur de somme en une phrase.

4. Pour une boucle `while` limitée à trois essais, nommer un variant et expliquer l'arrêt.

5. Quel comportement as-tu choisi pour une liste vide, et pourquoi ?

Mon auto-évaluation

Je peux...	Pas encore	Avec aide	Seul	Expliquer
Justifier le choix <code>for/while</code>	☐	☐	☐	☐
Sécuriser compteur et accumulateur	☐	☐	☐	☐
Traiter une liste vide	☐	☐	☐	☐
Énoncer invariant et variant	☐	☐	☐	☐
Écrire les tests avant le code	☐	☐	☐	☐

Bilan personnel

Le bug le plus instructif : _____

Le contrôle que je rendrai systématique : _____

Une preuve d'autonomie conservée aujourd'hui : _____